Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Démarrer PHP - Cours + TP progressifs

Pré-requis et environnement

Ce que fait PHP (en 30 secondes)

- **JavaScript** s'exécute principalement **dans le navigateur**.
- **PHP** s'exécute **sur le serveur** : il prépare une réponse (souvent du HTML) et l'envoie au navigateur.
- Le navigateur reçoit du **HTML/CSS/JS**. Il ne "voit" pas ton code PHP.

Lancer PHP en local (2 méthodes)

1. **Serveur PHP intégré** (rapide, parfait pour apprendre)

- Dans un dossier de projet :
  - `php -S 127.0.0.1:8000`
- Puis ouvre `http://127.0.0.1:8000`

2. **Stack type Apache/Nginx** (plus proche prod)

- Pratique quand on aborde ensuite la config, les virtual hosts, etc.

Organisation de projet recommandée (simple)

```
php-intro/
  index.php
  pages/
    conditions.php
    tableaux.php
    fonctions.php
    dates.php
    formulaire.php
    fichiers.php
  data/
    compteur.txt
    messages.txt
  public/
    css/
      style.css
```

> Astuce : garde tes fichiers accessibles par le serveur (public) et tes données dans un dossier séparé.

---

1 Structure d'un fichier PHP

Le bloc PHP

Un script PHP commence par `<?php` et se termine (optionnellement) par `?>`.

Recommandation : **ne pas fermer** `?>` quand le fichier est uniquement du PHP (évite des bugs d'espaces).

Exemple `index.php` :

```
<?php
echo "Bonjour le monde !";
```

PHP dans du HTML

On peut mélanger :

```
<?php $prenom = "Ada"; ?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Test PHP</title>
</head>
<body>
  <h1>Bienvenue</h1>
  <p>Bonjour <?= $prenom ?></p>
</body>
</html>
```

- `<?= ... ?>` est un raccourci de `<?php echo ...; ?>`

---

2 Affichage et débogage

Afficher

- `echo` : le plus courant
- `print` : similaire, rarement utile

```
echo "Hello";
echo "<br>";
echo "World";
```

Déboguer

```
var_dump($variable); // affiche type + valeur
print_r($tableau);   // lisible pour tableaux
```

Conseil : pour un tableau dans du HTML, affiche dans un `<pre>` :

```
echo "<pre>";
print_r($tableau);
echo "</pre>";
```

---

3 Variables, types et conversions

Variables

- Toujours préfixées par `$`
- Typage dynamique : le type dépend de la valeur
- Bonne pratique : initialiser

```
$age = 25;
$prenom = "Alice";
$estActif = true;
$prix = 19.99;
$note = null;
```

Comparaisons

- `==` compare les valeurs (conversion implicite possible)
- `===` compare valeur + type (recommandé)

Exemple :

```
$val = "0";
var_dump($val == 0);  // true (conversion)
var_dump($val === 0); // false (types différents)
```

Conversions utiles

```
$n = (int) "12";     // cast
$f = (float) "12.5";
$s = (string) 123;
```

---

4 Conditions

if / elseif / else

```
$age = 17;

if ($age >= 18) {
  echo "Majeur";
} elseif ($age >= 16) {
  echo "Presque";
} else {
  echo "Mineur";
}
```

switch (quand on teste une seule variable)

```
$jour = "Lundi";

switch ($jour) {
  case "Lundi":
    echo "Début de semaine";
    break;
  case "Vendredi":
    echo "Bientôt le week-end";
    break;
  default:
    echo "Jour normal";
}
```

Ternaire (à utiliser avec parcimonie)

```
$etat = ($age >= 18) ? "majeur" : "mineur";
```

Conseil lisibilité : garde le ternaire pour des cas simples.

---

5 Boucles

while

```
$i = 1;
while ($i <= 3) {
  echo "i = $i<br>";
  $i++;
}
```

for

```
for ($i = 1; $i <= 3; $i++) {
  echo "i = $i<br>";
}
```

foreach (indispensable avec les tableaux)

```
$fruits = ["Pomme", "Poire", "Banane"];

foreach ($fruits as $fruit) {
  echo $fruit . "<br>";
}
```

Avec clé + valeur :

```
$notes = ["Alice" => 15, "Bob" => 12];

foreach ($notes as $nom => $note) {
  echo "$nom : $note<br>";
}
```

---

6 Tableaux

Tableaux indexés

```
$couleurs = ["red", "blue", "white"];
echo $couleurs[2]; // white
```

Tableaux associatifs

```
$couleurs = [
  "rouge" => "#ff0000",
  "bleu"  => "#3366ff",
  "blanc" => "#ffffff",
];
echo $couleurs["bleu"];
```

Tableaux multi-dimensionnels

```
$personnes = [
  ["prenom" => "Ada", "age" => 36],
```

```
    ["prenom" => "Alan", "age" => 41],
  ];

  echo $personnes[0]["prenom"]; // Ada
```

Fonctions utiles sur les tableaux

- `count($t)` : taille
- `in_array($v, $t)` : contient ?
- `array_key_exists($k, $t)` : clé existe ?
- `array_push($t, $v)` ou `$t[] = $v`
- `array_map`, `array_filter` : très utiles (niveau CDA)

Exemple `array_filter` :

```
  $notes = [8, 12, 15, 4];
  $reussites = array_filter($notes, fn($n) => $n >= 10);
```

---

7 Fonctions

Définir une fonction

```
  function bonjour(string $prenom): string
  {
    return "Bonjour $prenom";
  }

  echo bonjour("Dave");
```

Paramètres optionnels

```
  function saluer(string $prenom, string $titre = "M."): string
  {
    return "Bonjour $titre $prenom";
  }
```

Bon réflexe : séparer "calcul" et "affichage"

- Une fonction retourne une valeur
- L'affichage se fait ailleurs avec `echo`

---

8 Dates et heures

Fuseau horaire

En France :

```
date_default_timezone_set("Europe/Paris");
```

Timestamp

```
echo time(); // secondes depuis 01/01/1970
```

DateTime (recommandé)

```
$now = new DateTime();
echo $now->format("d/m/Y H:i");
```

Ajouter un délai

```
$dt = new DateTime();
$dt->modify("+7 days");
echo $dt->format("d/m/Y");
```

---

9 Formulaires HTML et données serveur

GET vs POST

- GET : données dans l'URL (recherche, filtres)
- POST : données dans le corps (création, envoi de formulaire)

Lire les données côté PHP (bon réflexe)

1. Vérifier la méthode :

   ```
   if ($_SERVER["REQUEST_METHOD"] === "POST") {
   // ...
   }
   ```

2. Récupérer en sécurisant l'absence de clé :

   ```
   $prenom = $_POST["prenom"] ?? "";
   $nom    = $_POST["nom"] ?? "";
   ```

3. Échapper à l'affichage HTML (anti-XSS) :

```
echo htmlspecialchars($prenom, ENT_QUOTES, "UTF-8");
```

Exemple complet : `pages/formulaire.php`

```
<?php
date_default_timezone_set("Europe/Paris");

$prenom = "";
$nom = "";
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
  $prenom = trim($_POST["prenom"] ?? "");
  $nom    = trim($_POST["nom"] ?? "");

  if ($prenom === "" || $nom === "") {
    $message = "Merci de renseigner prénom et nom.";
  } else {
    $message = "Bonjour " . htmlspecialchars($prenom, ENT_QUOTES, "UTF-8")
             . " " . htmlspecialchars($nom, ENT_QUOTES, "UTF-8") . " !";
  }
}
?>
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Formulaire</title>
</head>
<body>
  <h1>Formulaire</h1>

  <?php if ($message !== ""): ?>
    <p><strong><?= $message ?></strong></p>
  <?php endif; ?>

  <form method="post">
    <label>Prénom <input type="text" name="prenom" value="<?=
htmlspecialchars($prenom, ENT_QUOTES, "UTF-8") ?>"></label><br>
    <label>Nom <input type="text" name="nom" value="<?=
htmlspecialchars($nom, ENT_QUOTES, "UTF-8") ?>"></label><br>
    <button type="submit">Envoyer</button>
  </form>
</body>
</html>
```

10 Manipulation de fichiers

Ouvrir, écrire, fermer

```
$fp = fopen(__DIR__ . "/../data/messages.txt", "a");
if ($fp === false) {
  die("Impossible d'ouvrir le fichier.");
}

fwrite($fp, "Hello\n");
fclose($fp);
```

Lire ligne par ligne

```
$fp = fopen(__DIR__ . "/../data/messages.txt", "r");
if ($fp === false) {
  die("Impossible d'ouvrir le fichier.");
}

while (!feof($fp)) {
  $ligne = fgets($fp);
  if ($ligne === false) break;
  echo htmlspecialchars($ligne, ENT_QUOTES, "UTF-8") . "<br>";
}
fclose($fp);
```

Lire tout d'un coup (fichier raisonnable)

- `file()` : retourne un tableau de lignes
- `file_get_contents()` : retourne une chaîne

---

11 Upload de fichiers (introduction, version sécurisée)

Côté HTML

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="fichier">
  <button type="submit">Envoyer</button>
</form>
```

Côté PHP (principes de base)

- Vérifier l'erreur `$_FILES["fichier"]["error"]`
- Limiter la taille

- Vérifier le type réel (pas seulement l'extension)
- Renommer côté serveur
- Ne jamais exécuter un fichier uploadé

Exemple `upload.php` (simplifié et raisonnable) :

```
<?php
$uploadsDir = __DIR__ . "/../data/uploads";
if (!is_dir($uploadsDir)) {
  mkdir($uploadsDir, 0775, true);
}

if ($_SERVER["REQUEST_METHOD"] === "POST") {
  $f = $_FILES["fichier"] ?? null;

  if ($f === null || $f["error"] !== UPLOAD_ERR_OK) {
    die("Upload échoué.");
  }

  if ($f["size"] > 2 * 1024 * 1024) { // 2 Mo
    die("Fichier trop volumineux.");
  }

  $tmp = $f["tmp_name"];
  $mime = mime_content_type($tmp);

  $allowed = ["image/jpeg", "image/png", "application/pdf"];
  if (!in_array($mime, $allowed, true)) {
    die("Type de fichier non autorisé.");
  }

  $ext = match ($mime) {
    "image/jpeg" => "jpg",
    "image/png" => "png",
    "application/pdf" => "pdf",
    default => "bin",
  };

  $newName = bin2hex(random_bytes(16)) . "." . $ext;
  $dest = $uploadsDir . "/" . $newName;

  if (!move_uploaded_file($tmp, $dest)) {
    die("Impossible de déplacer le fichier.");
  }

  echo "OK, fichier enregistré : " . htmlspecialchars($newName, ENT_QUOTES,
"UTF-8");
}
```